EXPERIMENT :-6

CODE:-

```
#include <stdio.h>

int board[20][20];
int n;

int safe(int row, int col) {
  int i, j;

  // Check column
  for (i = 0; i < row; i++)
    if (board[i][col])
      return 0;

  // Check upper-left diagonal
  for (i = row - 1, j = col - 1;
     i >= 0 && j >= 0; i--, j--)
    if (board[i][j])
      return 0;

  // Check upper-right diagonal
  for (i = row - 1, j = col + 1;
     i >= 0 && j < n; i--, j++)
    if (board[i][j])
      return 0;
```

```
    return 1;
}

int solve(int row) {
    int col;

    if (row == n)
        return 1;

    for (col = 0; col < n; col++) {
        if (safe(row, col)) {
            board[row][col] = 1;

            if (solve(row + 1))
                return 1;

            board[row][col] = 0;   // Backtrack
        }
    }

    return 0;
}

int main() {
    int i, j;

    printf("Enter number of queens: ");
    scanf("%d", &n);
```

```
  if (solve(0)) {
    printf("\nSolution:\n");

    for (i = 0; i < n; i++) {
      for (j = 0; j < n; j++)
        printf("%d ", board[i][j]);

      printf("\n");
    }
  } else {
    printf("No solution exists.");
  }

  return 0;
}
```

Example Input

```
Enter number of queens: 4
```

Output

```
Solution:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
```

Here **1** = **Queen** and **0** = **Empty position**.